

STOCHASTIC SCANNER PRO

v1.0

Monte Carlo · GARCH(1,1) · t-student · Calibración

QUÉ ES ESTE SISTEMA: Un motor de proyecciones estocásticas independiente basado en matemática financiera estándar. Calcula probabilidades calibradas de movimientos de precio usando 10,000 simulaciones Monte Carlo por ticker, con volatilidad GARCH(1,1) y distribución t-student para colas pesadas.

QUÉ NO ES: No es "cuántico", no es IA, no es predicción exacta del futuro. No es consejo financiero ni garantía de resultados.

Contenido

1. Filosofía y alcance del sistema
2. Arquitectura técnica
3. Pipeline: Datos → SPE → Snapshot
4. Interpretación de las tarjetas del dashboard
5. Validación de calibración (crítico)
6. Despliegue a GitHub Pages
7. Personalización del universo de tickers
8. Limitaciones conocidas
9. Glosario de términos estadísticos

1. Filosofía y alcance del sistema

Principio fundamental: honestidad matemática

Este sistema fue construido bajo un principio simple — comunicar con precisión lo que hace y lo que no hace. En finanzas cuantitativas hay mucho marketing engañoso: "quantum AI", "predicción con 95% de exactitud", "indicador secreto de Wall Street". Nada de eso existe. Si existiera, los fondos que lo descubrieran ya habrían capturado todo el dinero del mercado.

Stochastic Scanner Pro hace exactamente una cosa bien: **estimar probabilidades calibradas de escenarios futuros de precio usando simulación Monte Carlo**. Nada más, nada menos.

Qué hace el sistema:

- Descarga 2 años de precios diarios desde Yahoo Finance
- Ajusta distribución t-student a los retornos log (con chequeo de tamaño muestral)
- Ajusta modelo GARCH(1,1) por Maximum Likelihood Estimation
- Clasifica el régimen de mercado actual (5 categorías)
- Simula 10,000 trayectorias de precio con volatilidad dinámica
- Calcula probabilidades, intervalos de confianza, VaR y CVaR
- Proyecta a 5 horizontes: 1w, 2w, 1m, 3m, 6m
- Valida su propia calibración con walk-forward backtesting

Qué NO hace:

- **No predice el futuro con exactitud.** Da probabilidades, no certezas.
- **No incorpora fundamentales** (P/E, ROE, earnings). Usa solo precios.
- **No modela eventos corporativos** (splits, M&A, earnings date).
- **No sustituye análisis humano.** Es una herramienta, no un oráculo.
- **No es consejo financiero.** Cualquier decisión de inversión es tuya.

2. Arquitectura técnica

Componentes principales:

Archivo	Responsabilidad
stochastic_projector.py	Motor SPE: GARCH + t-student + MC + regímenes
build_data.py	Pipeline: fetch yfinance → indicadores → SPE → snapshot.json
calibration_test_offline.py	Valida modelo con datos sintéticos GARCH (offline)
calibration_backtest.py	Valida modelo con walk-forward sobre datos reales
index.html	Dashboard web: lee snapshot.json y renderiza tarjetas
generate_demo_snapshot.py	Genera snapshot de ejemplo sin depender de Yahoo

Stack tecnológico:

Capa	Tecnología
Datos	yfinance (Yahoo Finance unofficial API)
Cálculo numérico	numpy, pandas, scipy
Optimización MLE (GARCH)	scipy.optimize (L-BFGS-B)
Frontend	HTML + vanilla JavaScript (sin frameworks)
Hosting	GitHub Pages (archivos estáticos)
Actualización	GitHub Actions (diaria, 21:30 UTC)

Flujo de datos:

El flujo es unidireccional y bien definido:

```
>yfinance API → build_data.py → snapshot.json → index.html (browser)
```

No hay backend. No hay base de datos. El snapshot.json es el único archivo de estado. Esto hace el sistema simple de entender, simple de desplegar, y simple de validar.

3. Pipeline: Datos → SPE → Snapshot

Paso 1 — Fetch de datos

Por cada ticker del universo, se descargan 2 años de precios diarios (Open, High, Low, Close, Volume). Si el ticker tiene menos de 250 días de historia (1 año), se omite.

Paso 2 — Indicadores básicos

Se calculan cuatro indicadores técnicos — los únicos que SPE necesita para detectar régimen de mercado:

Indicador	Uso
ATR (14)	Detectar expansión/contracción de volatilidad
RSI (14)	Contexto del momentum (el modelo no lo usa como señal directa)
ADX proxy	Fuerza de la tendencia (usado para clasificar regímenes)
Volumen relativo	Detectar eventos inusuales (alta actividad)

Paso 3 — Stochastic Projection Engine

El corazón del sistema. Proceso interno:

- 3.1 Convertir precios a retornos log: $r_t = \ln(P_t / P_{t-1})$
- 3.2 Ajustar distribución t-student a los retornos via `scipy.stats.t.fit()`. Si hay menos de 60 días, cae a distribución normal (más robusta con poca data).
- 3.3 Ajustar GARCH(1,1) via MLE: $\sigma^2(t) = \omega + \alpha \cdot r^2(t-1) + \beta \cdot \sigma^2(t-1)$. Si el ajuste falla o diverge, cae a EWMA (RiskMetrics, $\lambda=0.94$).
- 3.4 Forecast de volatilidad para cada día del horizonte usando GARCH.
- 3.5 Simular 10,000 trayectorias: para cada una, sample 21 innovations t-student, escalarlas por $\sigma(t)$ del forecast GARCH, acumular como log returns, exponenciar para obtener precio terminal.
- 3.6 Calcular estadísticas sobre los 10,000 precios terminales: media, mediana, percentiles 2.5%, 16%, 84%, 97.5%, VaR(5%), CVaR(5%).
- 3.7 Clasificar régimen y calcular ensemble con target técnico.

Paso 4 — Guardar snapshot

El resultado se serializa a `data/snapshot.json` con la siguiente estructura por ticker: precio actual, probabilidades, intervalos de confianza, métricas de riesgo, régimen, multi-horizon, últimos 60 días para el mini-chart.

4. Interpretación de las tarjetas del dashboard

Cada ticker se renderiza como una tarjeta con varias zonas. Veamos cómo leer cada una correctamente.

Probabilidad UP (zona superior izquierda):

Ejemplo: "Prob UP: 62.4%"

Significado CORRECTO: Entre las 10,000 trayectorias simuladas, el 62.4% terminaron por encima del precio actual al horizonte seleccionado.

Significado INCORRECTO: "El precio va a subir con 62.4% de seguridad." Esto malinterpreta la naturaleza probabilística. En un modelo bien calibrado, eventos predichos con 62% deberían ocurrir ~62% de las veces *en promedio sobre muchas observaciones*. Cualquier trade individual puede ganar o perder.

Target esperado (zona superior derecha):

Es la **media** de los 10,000 precios terminales simulados. No es un "precio objetivo" en el sentido tradicional — es el centro de una distribución de escenarios. El upside % se calcula vs el precio actual.

Mini-chart de 60 días:

Muestra la tendencia reciente. La línea verde/roja indica dirección de los últimos 60 días vs el inicio. La línea punteada amarilla marca el target esperado. El rectángulo azul a la derecha representa el CI 68%.

Intervalo de confianza 68%:

El marcador amarillo muestra la posición del precio actual dentro del CI. En un modelo calibrado, hay aproximadamente 68% de probabilidad que el precio al horizonte caiga dentro de este rango.

Probabilidades TP (zona media):

Prob +3%, +5%, +10% son las probabilidades estimadas de que el precio alcance cada nivel al horizonte. Útil para evaluar targets específicos de take-profit.

VaR y CVaR (zona de riesgo):

VaR 95%: En el peor 5% de escenarios, el precio al horizonte será $\geq$ este valor. Es el percentil 5.

CVaR 95%: El PROMEDIO de todos los escenarios en el peor 5%. Siempre es peor (más bajo) que VaR. Mide la severidad de los escenarios de cola, no solo la probabilidad.

Uso práctico: Si tu límite de pérdida por trade es \$500, calcula el tamaño de posición usando CVaR (no precio actual). Ejemplo: precio actual \$100, CVaR \$88 $\rightarrow$ pérdida esperada en escenarios malos = \$12/acción $\rightarrow$ máximo 41 acciones para limitar riesgo a \$500.

Multi-horizon strip:

5 mini-celdas con probabilidades estimadas a 1 semana, 2 semanas, 1 mes, 3 meses y 6 meses. Útil para ver cómo cambia la incertidumbre con el tiempo — usualmente el CI se expande y las probabilidades convergen hacia 50%.

Badges de calidad:

Badge	Significado
★ reliable	≥ 60 días de datos, t-student fit válido
■ limited data	< 60 días, modelo cae a normal (menos preciso)
GARCH	Volatilidad dinámica GARCH(1,1) convergió — usar con confianza
EWMA	GARCH falló, usando fallback RiskMetrics — aún robusto
STRONG_TREND / TREND	Mercado direccional, tendencia clara
COMPRESSION	Volatilidad contrayendo — posible breakout próximo
RANGE	Mercado lateral, sin tendencia clara
HIGH_VOL	Volatilidad extrema — reducir tamaño de posición

5. Validación de calibración (CRÍTICO)

LO MÁS IMPORTANTE DE ESTE MANUAL. Antes de confiar en cualquier probabilidad que da el sistema, debes ejecutar el backtesting de calibración. Un modelo "bien calibrado" dice 60% cuando la realidad empírica es 60%. Un modelo mal calibrado puede decir 80% cuando la realidad es 50% — peor que no tener modelo, porque te da falsa confianza.

Dos tests incluidos en el sistema:

Test 1 — Offline (sin internet):

```
> python scripts/calibration_test_offline.py
```

Genera 8-15 series sintéticas con GARCH(1,1) + t-student (imitando mercados reales), corre walk-forward sobre cada una, y reporta métricas. Útil como sanity check rápido. Tiempo: ~3 minutos.

Test 2 — Con datos reales (requiere internet):

```
> python scripts/calibration_backtest.py --tickers SPY,QQQ,AAPL --years 5
```

Descarga 5 años de datos reales de Yahoo, y valida el modelo sobre ellos. Este es el test que realmente importa — te dice si el modelo funciona en tu universo específico de tickers. Tiempo: ~10-30 minutos dependiendo del número de tickers.

Métricas reportadas y cómo interpretarlas:

Métrica	Valor ideal	Interpretación
ECE (Expected Calibration Error)	< 0.03	Excelente
ECE	0.03 - 0.06	Bueno, usable con normal cautela
ECE	0.06 - 0.10	Aceptable, no confiar ciegamente
ECE	> 0.10	Mala calibración — no usar probabilidades
Brier score	< 0.22	Mejor que aleatorio
Brier score	≥ 0.25	Equivalente a lanzar moneda
Directional accuracy	> 52%	Genera alpha direccional
Directional accuracy	≈ 50%	Sin edge predictivo

El reliability diagram:

La salida incluye una tabla de bins que muestra, para cada rango de probabilidad estimada (ej: [0.55, 0.60)), cuántas predicciones cayeron ahí y qué porcentaje realmente se cumplió. Un modelo perfecto tendría `avg_pred` ≈ empírica en cada bin.

```
> [0.50,0.55) 127 0.525 0.543 0.018 excellent
```

```
> [0.55,0.60) 98 0.575 0.581 0.006 excellent
```

Recomendación: ejecutar calibración cada 3 meses

Los mercados cambian. Un modelo calibrado en 2023 puede desviarse en 2024 si el régimen de volatilidad cambia drásticamente. Agregar al calendario: correr `calibration_backtest.py` cada trimestre y verificar ECE.

6. Despliegue a GitHub Pages

Pasos básicos:

1. Crear repo en GitHub (público) y subir el contenido del ZIP.
2. Settings → Pages → Source = 'Deploy from a branch', Branch = 'main', Folder = '/' (root).
3. Esperar 1-2 minutos. Tu dashboard estará en <https://TU-USER.github.io/TU-REPO/>
4. Opcional: Habilitar GitHub Actions (Settings → Actions → allow all). La Action refreshea el snapshot diariamente.

Primer snapshot:

El ZIP incluye un snapshot de ejemplo con datos sintéticos (para que puedas ver la UI funcionando sin configurar nada). Para datos reales:

```
> pip install -r requirements.txt

> python scripts/build_data.py --out-dir data

> git add data/snapshot.json && git commit -m 'first real snapshot' && git push
```

Estructura del repo:

```
> mi-repo/

> ■■■ index.html ← lo que ve el usuario

> ■■■ data/snapshot.json ← generado automáticamente

> ■■■ scripts/ ... ← Python backend

> ■■■ requirements.txt

> ■■■ .github/workflows/refresh.yml
```

7. Personalización del universo de tickers

El universo por defecto incluye ~100 tickers agrupados en 6 categorías (US Large Cap, US Tech, US Finance, ETFs & Indexes, International, LATAM). Para personalizar:

Opción 1 — Línea de comandos (temporal):

```
> python scripts/build_data.py --tickers AAPL,MSFT,NVDA,TSLA
```

Opción 2 — Editar build_data.py (permanente):

Abre `scripts/build_data.py` y modifica el diccionario `TICKER_GROUPS`. Puedes añadir/quitar grupos, añadir/quitar tickers.

Consideraciones:

- Cada ticker toma ~1-2 segundos de cómputo SPE + tiempo de red Yahoo
- 100 tickers → ~3-5 minutos total
- 500 tickers → ~15-25 minutos (mejor usar GitHub Action nocturna)
- Tickers ilíquidos (<60 días data) son omitidos automáticamente
- Yahoo puede rate-limitar si haces muchos requests muy rápido

8. Limitaciones conocidas

Un modelo honesto conoce sus límites. Estos son los de Stochastic Scanner Pro.

1. Asume que el futuro se parece al pasado:

Monte Carlo usa la distribución histórica de returns para generar escenarios futuros. Un cambio de régimen (guerra, pandemia, cambio de política monetaria) invalida temporalmente esta asunción hasta que haya datos post-evento.

2. No captura eventos corporativos:

Earnings, splits, dividendos especiales, M&A, cambios de CEO — nada de esto entra al modelo. Un ticker con earnings en 3 días tendrá proyecciones que IGNORAN ese catalizador binario específico.

3. Calibración degradada en crisis:

En marzo 2020, octubre 2008, las correlaciones colapsaron y las distribuciones cambiaron rápido. GARCH ayuda (adapta volatilidad rápido) pero no es mágico. La calibración puede empeorar temporalmente durante esos períodos.

4. No modela liquidez ni slippage:

Las probabilidades asumen que puedes comprar/vender al precio de cierre. En acciones de baja liquidez, el slippage real puede consumir el edge estadístico.

5. Sesgo de supervivencia en la selección de tickers:

El universo incluye tickers que existen hoy. Los tickers que quebraron no están. Esto sesga cualquier análisis agregado hacia los "supervivientes".

6. Yahoo Finance no es fuente oficial:

yfinance es un wrapper no-oficial que puede tener datos inexactos, especialmente para tickers internacionales o con splits recientes. Para uso profesional, considera un proveedor pagado (IEX, Polygon, Alpaca).

9. Glosario de términos estadísticos

Monte Carlo: Método de simular múltiples trayectorias aleatorias para estimar distribuciones de resultados. Inventado por Stanislaw Ulam en Los Álamos (1946).

GARCH(1,1): Generalized Autoregressive Conditional Heteroskedasticity. Modela volatilidad que cambia con el tiempo (clusters). $\sigma^2(t) = \omega + \alpha \cdot r^2(t-1) + \beta \cdot \sigma^2(t-1)$.

EWMA: Exponentially Weighted Moving Average. Estándar de RiskMetrics (JP Morgan, 1996) para estimar volatilidad con decaimiento exponencial.

t-student: Distribución de probabilidad similar a la normal pero con colas más pesadas. Modela mejor los retornos financieros reales (eventos extremos más frecuentes que bajo la normal).

VaR 95%: Value at Risk. 'En el peor 5% de los escenarios, ¿cuál es el valor?' Usado como umbral de pérdida aceptable.

CVaR 95%: Conditional VaR (también llamado Expected Shortfall). 'Si caemos en el peor 5%, ¿cuál es el promedio?' Siempre peor que VaR.

Brier score: Métrica de calidad de probabilidades predichas. $\text{Brier} = \text{media}((\text{pred} - \text{real})^2)$. Rango [0, 1]. Menor = mejor. 0 = perfecto, 0.25 = aleatorio.

ECE: Expected Calibration Error. Promedio ponderado de las desviaciones entre probabilidad predicha y frecuencia empírica en cada bin. Menor = mejor calibrado.

Kurtosis: Medida de qué tan pesadas son las colas de una distribución. Kurtosis normal = 0. Si kurtosis > 0, colas más pesadas (eventos extremos más comunes).

Skewness: Asimetría de una distribución. Positivo = cola larga hacia arriba. Negativo = cola larga hacia abajo. Cero = simétrica.

MLE: Maximum Likelihood Estimation. Método para ajustar parámetros de un modelo maximizando la probabilidad de observar los datos reales bajo ese modelo.

CI 68% / CI 95%: Confidence Intervals. Rangos donde la probabilidad estimada de que el precio caiga es 68% o 95% respectivamente. Corresponden a ± 1 y ± 2 desviaciones estándar bajo normal.

Stochastic Scanner Pro v1.0

Independent stochastic projection system

No marketing · No pseudoscience · Calibrated probabilities